GUIA DIDÁTICO

Prompt Vault
por dentro

Python, arquitetura hexagonal e mensageria explicados com um projeto de verdade



- **domain** regras de negócio
- **app** casos de uso
- **port** contratos
- **adapter** tradutores
- **infra** tecnologia configurada

Material de estudo gerado a partir do código implementado e testado

10 de novembro de 2026

Sumário

Clique em qualquer item para ir direto à página.

0. Antes de começar 4

Como estudar com este guia 4

Mapa de leitura 4

1. O problema e a solução 6

1.1 Prompts, e por que versioná-los 6

1.2 O que o sistema faz 6

1.3 Sem tela, sem HTTP: o sistema conversa por mensagens 7

1.4 O mapa do projeto 7

2. Python essencial 9

2.1 O ambiente: uv, pyproject.toml e uv.lock 9

2.2 Módulos, pacotes e imports 10

2.3 Tipos: dizendo o que cada coisa é 10

2.4 Classes, dataclasses e enums 10

2.5 Protocol: contratos sem herança 11

2.6 async / await: o garçom que atende várias mesas 11

2.7 Context managers: “with” e “async with” 12

2.8 Exceções: quando algo não pode continuar 12

3. Arquitetura hexagonal 14

3.1 A ideia: proteger as regras da tecnologia 14

3.2 Os cinco blocos 15

3.3 A Regra de Dependência 15

3.4 Por que vale a pena (e o que custa) 16

3.5 Os padrões que você vai encontrar 16

4. Domain: o coração 17

4.1 Modelagem rica, não anêmica 17

4.2 O agregado: Prompt e suas versões 17

4.3 Versões imutáveis e o ponteiro da versão ativa 18

4.4 O ciclo de vida: uma máquina de estados 18

4.5 Invariantes: o que é sempre verdade 20

4.6 Eventos de domínio: fatos ocorridos 20

4.7 O diff e o padrão Strategy 21

4.8 Uma lição que o teste ensinou 22

5. Ports e Application 23

5.1 Ports: o que o núcleo exige do mundo 23

5.2 Unit of Work: a transação “tudo ou nada” 24

5.3 Casos de uso: a orquestra 24

5.4 Anatomia de um caso de uso 24

5.5 Injeção de dependência e o poder do “falso” 25

6. Adapters e Infra 26

6.1 Infra: configurando as tecnologias 26

6.2 Persistência: SQLAlchemy e SQLite 26

6.3 Mensageria: o adapter do NATS 29

6.4 Logging estruturado: structlog + rich 32

6.5 O main.py: a raiz de composição 32

7. Uma mensagem de ponta a ponta 34

7.1 Passo a passo 34

7.2 O que cada observador enxerga 35

8. Verificando que funciona 36

8.1 Verificar em camadas 36

8.2 O smoke test 36

8.3 Outras verificações 37

8.4 Lições do caminho 37

9. Mão na massa 38

9.1 Subindo o ambiente 38

9.2 Receitas com o NATS CLI 38

9.3 Exercícios 38

9.4 Próximos passos naturais 39

Glossário 40

Mapa de arquivos 41

Para ir além 42

Livros e artigos 42

Documentação oficial 42

Respostas das pausas 43

0 Antes de começar

Para quem é este guia e como estudá-lo

*Bem-vindo à aula. Este guia foi escrito para quem **nunca programou em Python** (ou programou pouco) e quer entender, de verdade, o projeto **Prompt Vault**: o que cada pedaço faz, por que está onde está e como os pedaços conversam entre si. Em vez de explicar tecnologias no vazio, vamos usar o próprio projeto como laboratório: **todos os trechos de código que você verá foram copiados dos arquivos reais**, e todos os resultados de execução foram obtidos rodando o sistema.*

Como estudar com este guia

1. Leia na ordem. Cada capítulo começa com uma **pergunta-guia** e fecha com um **resumo** (ou uma síntese); os capítulos 2 a 6 trazem também uma **pausa para fixar** (as respostas estão no Apêndice D).
2. Quando o texto citar um arquivo, abra-o no editor ao lado. Ver o código “de verdade” fixa muito mais do que ler sobre ele.
3. Rode os comandos do Capítulo 9. Mandar uma mensagem e ver o sistema responder transforma conceito em experiência.
4. Não se preocupe em decorar. O objetivo é saber *onde olhar* e *por que* cada coisa existe.

Mapa de leitura

Capítulo	A pergunta que ele responde
1 · O problema e a solução	Para que existe o sistema e como alguém conversa com ele?
2 · Python essencial	Que partes da linguagem preciso saber para ler este código?
3 · Arquitetura hexagonal	Por que o código está dividido em cinco pastas e quem pode falar com quem?
4 · Domain	Onde moram as regras do negócio e como elas se protegem?
5 · Ports e Application	Como um pedido vira um caso de uso sem depender de tecnologia?
6 · Adapters e Infra	Como o núcleo se conecta ao NATS, ao SQLite e ao terminal?
7 · Uma mensagem de ponta a ponta	O que acontece, passo a passo, quando alguém altera um prompt?
8 · Verificando que funciona	Como sabemos que tudo isso realmente funciona?
9 · Mão na massa	Como eu mesmo uso, exploro e estendo o projeto?

As caixas coloridas

CONCEITO	Uma ideia ou termo novo, explicado com calma.
ANALOGIA	Uma comparação com o mundo cotidiano para ancorar a ideia.
NO PROJETO	Onde aquela ideia aparece, concretamente, no código do Prompt Vault.
ATENÇÃO	Uma armadilha comum ou uma limitação honesta do projeto.
LIÇÃO APRENDIDA	Algo que deu errado durante o desenvolvimento e o que aprendemos com isso.
PAUSA PARA FIXAR	Três perguntas no fim de cada capítulo (respostas no Apêndice D).

1 O problema e a solução

Para que existe o Prompt Vault e como alguém conversa com ele

Pergunta-guia: *para que existe este sistema e como alguém o usa, se ele não tem tela nem site?*

1.1 Prompts, e por que versioná-los

Um **prompt** é o texto de instrução que enviamos a um modelo de linguagem (um *LLM*, como o Claude): "resuma o texto abaixo em três tópicos, em tom formal". Trocar uma única palavra pode mudar bastante a resposta. Quem trabalha com LLMs acaba com dezenas de prompts que evoluem semana a semana — e surgem perguntas incômodas: *qual versão funcionava ontem? quem mudou o quê? posso voltar atrás?*

> **ANALOGIA · O histórico do documento**
> Pense no histórico de versões de um Google Docs, ou nos *commits* do Git. Ninguém apaga o passado: apenas acrescenta novos capítulos e escolhe qual deles está "valendo". O Prompt Vault faz exatamente isso, só que especializado em prompts.

1.2 O que o sistema faz

Capacidade	Subject NATS	Comportamento
Criar um prompt	`prompt.create`	Nasce **ativo**, com a versão 1 já ativa.
Criar nova versão	`prompt.version.create`	Acrescenta uma versão; só a ativa se você pedir (*activate*).
Alterar o conteúdo	`prompt.content.update`	Nunca edita: cria uma **nova versão e a ativa**.
Ativar uma versão	`prompt.version.activate`	Escolhe qual versão está "valendo" — inclusive uma antiga (*rollback*).
Apagar (soft delete)	`prompt.delete`	Esconde o prompt, mas guarda tudo. Pode ser recuperado.
Recuperar	`prompt.restore`	Volta ao estado em que o prompt estava antes de ser apagado.
Consultar	`prompt.get`, `prompt.versions.list`, `prompt.list.active`, `prompt.list.deleted`	Perguntas que só leem — nunca alteram nada.
Comparar versões	`prompt.versions.compare`	Mostra linhas adicionadas, removidas e modificadas entre duas versões (*diff*).

CONCEITO · As regras do jogo

- **Toda alteração de conteúdo gera uma nova versão.** Nada é sobrescrito.
- **Apenas uma versão está ativa** por prompt, a qualquer momento.
- **O histórico nunca se perde.** Apagar é "soft": o prompt some das listas ativas, mas continua guardado.
- As regras acima vivem **no coração do sistema** (a camada *domain*), não espalhadas pelo código.

1.3 Sem tela, sem HTTP: o sistema conversa por mensagens

Repare numa decisão de projeto importante: **não há site, nem API HTTP**. O Prompt Vault é um serviço que fica "escutando" mensagens num barramento chamado **NATS**. Quem quiser usá-lo envia uma mensagem para um *assunto* (*subject*), como `prompt.create`, e recebe a resposta de volta.

ANALOGIA · A central de rádio

Imagine uma central de rádio de táxi. Você não liga para um motorista específico: fala num *canal* ("canal 7, preciso de um carro") e quem atende aquele canal responde. O NATS é a central; os subjects são os canais; o Prompt Vault é a equipe que atende os canais `prompt.*`.

Veja uma conversa real, feita com o *NATS CLI* (uma ferramenta de linha de comando para falar com o NATS):

Terminal · saída real do sistema (a resposta, uma só linha, foi quebrada para caber)

```
$ nats request prompt.create '{"name":"cli","content":"ola"}' --raw
{"ok": true, "correlation_id": "25e14cde-cb7d-4f31-9fe5-29f8cce8cb65",
 "data": {"prompt_id": "f11c267d-3463-4e58-a7cb-f70f06d0c805"}}
```

A resposta é um **envelope** em JSON: `ok` diz se deu certo, `correlation_id` é uma etiqueta de rastreio (falaremos dela no Capítulo 6) e `data` traz o resultado — aqui, o identificador do prompt criado.

1.4 O mapa do projeto

O projeto foi construído em fases, de dentro para fora: primeiro o **domínio**, depois as **portas** e a **aplicação**, o **diff** entre versões e, por fim, os **adapters** e a **infraestrutura** que conectam tudo ao mundo real. São cerca de 2 mil linhas de Python, organizadas assim:

Estrutura de pastas

```
prompt-vault/
├── docker-compose.yml       servidor NATS local
├── pyproject.toml, uv.lock  dependências (gerenciadas com uv)
├── smoke_test.sh            teste de fumaça com o NATS CLI
└── src/
    ├── main.py              raiz de composição: liga tudo         (90 linhas)
    ├── domain/              regras de negócio puras               (605)
    ├── port/                contratos (Protocols)                  (57)
    ├── app/                 casos de uso                          (311)
    ├── adapter/
    │   ├── persistence/     SQLAlchemy + SQLite
    │   └── messaging/       NATS                      (833 no total)
    └── infra/               configuração das tecnologias          (150)
```

As tecnologias, em uma tabela

Tecnologia	Versão	Para que serve aqui
Python	3.13	A linguagem em que tudo foi escrito.
uv	0.13.7	Cria o ambiente, instala dependências e roda o programa.
NATS + *nats-py*	servidor 2.x · lib 2.16.0	O barramento de mensagens: a “porta de entrada” do sistema.
SQLite	embutido no Python	O banco de dados: um simples arquivo.
SQLAlchemy + *aiosqlite*	2.1.8 · 0.22.1	Traduz objetos Python em linhas de tabela, de forma assíncrona.
structlog	26.1.0	Registros (logs) estruturados, com campos pesquisáveis.
rich	15.0.0	Deixa a saída do terminal legível e colorida.
Docker Compose	—	Sobe o servidor NATS com um único comando.

RESUMO DO CAPÍTULO

- O Prompt Vault **versiona prompts**: cada mudança é uma versão nova; uma delas está ativa; nada se perde.
- Ele **não tem tela nem HTTP**: conversa por mensagens num barramento (NATS).
- O código tem cinco áreas (*domain, port, app, adapter, infra*) — o próximo passo é entender por quê. Antes, um pouco de Python.

2 Python essencial

A parte da linguagem de que você precisa para ler este projeto com tranquilidade

Pergunta-guia: *que recursos de Python aparecem no código e o que cada um significa?*

Você não precisa "saber Python" para acompanhar o projeto — só reconhecer meia dúzia de ideias que se repetem. Vamos vê-las na ordem em que aparecem, sempre ancoradas em código real.

2.1 O ambiente: uv, pyproject.toml e uv.lock

Programas Python dependem de bibliotecas de terceiros. Para que elas não se misturem com as de outros projetos, cada projeto tem seu próprio **ambiente virtual** (uma pasta `.venv` com uma "caixa" isolada de bibliotecas). O **uv** é a ferramenta que cria essa caixa e cuida dela.

pyproject.toml · a "ficha" do projeto

```
[project]
name = "prompt-vault"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.13"
dependencies = [
    "aiosqlite>=0.22.1",
    "nats-py>=2.16.0",
    "rich>=15.0.0",
    "sqlalchemy[asyncio]>=2.1.8",
    "structlog>=26.1.0",
]
```

Arquivo / comando	O que faz
`pyproject.toml`	Declara o nome do projeto, a versão mínima do Python e as **dependências**. Você edita (ou deixa o uv editar).
`uv.lock`	Registra as **versões exatas** de tudo que foi instalado, para qualquer pessoa reproduzir o mesmo ambiente. Gerado pelo uv: não se edita à mão.
`uv sync`	Instala exatamente o que o `uv.lock` manda.
`uv add pacote`	Adiciona uma dependência (atualiza `pyproject.toml` e `uv.lock`).
`uv run src/main.py`	Roda o programa dentro do ambiente do projeto.

LIÇÃO APRENDIDA · Uma dependência que faltava

Ao ligar o SQLAlchemy 2.1 no modo assíncrono, o programa falhou: ele exige a biblioteca *greenlet*, que a versão nova deixou de instalar sozinha. A correção foi pedir o "extra" `sqlalchemy[asyncio]` (veja na lista acima). Moral: o erro de importação diz exatamente o que falta — vale ler a mensagem até o fim.

2.2 Módulos, pacotes e imports

Um arquivo `.py` é um **módulo**. Uma pasta com um arquivo `__init__.py` é um **pacote** — um módulo que agrupa outros. Para usar algo de outro lugar, usamos `import`:

Três jeitos de importar (exemplos do projeto)

```
from domain import Prompt            # do pacote 'domain', traga a classe Prompt
from .status import PromptStatus      # ponto = 'o arquivo vizinho a este'
from ..events import PromptCreated    # dois pontos = 'a pasta acima'
```

NO PROJETO · Por que "uv run src/main.py" funciona?

Quando você executa um script, o Python coloca **a pasta desse script** na lista de lugares onde procura módulos. Como o script é `src/main.py`, a pasta `src/` entra na busca — e `import domain` encontra `src/domain/`. Para rodar qualquer outra coisa a partir da raiz do projeto, usamos `PYTHONPATH=src` para dar ao Python a mesma pista.

2.3 Tipos: dizendo o que cada coisa é

Em Python, você pode (e no projeto **sempre** fazemos) anotar o tipo esperado de cada valor. O Python não obriga isso na hora de rodar, mas ferramentas como o *pyright* conferem tudo antes — e o código vira sua própria documentação.

Escrita	Leitura em português
`name: str`	"name é um texto"
`versions: list[PromptVersion]`	"versions é uma lista de PromptVersion"
`active_version_number: int | None`	"é um número inteiro **ou** nada (None)" — aqui, "nenhuma versão ativa"
`def delete(self) -> None`	"delete é uma função que não devolve nada"
`async def get(...) -> Prompt | None`	"função assíncrona que devolve um Prompt ou None"

2.4 Classes, dataclasses e enums

Uma **classe** é um molde de objetos: reúne dados (*atributos*) e comportamentos (*métodos*). Dentro dela, `self` significa "este objeto aqui". Três variações aparecem o tempo todo:

- `@dataclass` — o "@" é um *decorador*: um selo colado na classe. Este gera automaticamente o construtor e a comparação, poupando código repetitivo.
- `frozen=True` — dentro do `@dataclass`, torna o objeto **imutável**: depois de criado, não pode mais mudar.
- `Enum` / `StrEnum` — uma lista fechada de opções válidas (como os estados de um prompt).
- `@property` — transforma um método em algo que se lê como atributo e que **não pode ser alterado de fora**.

src/domain/entities/prompt_version.py

```python
@dataclass(frozen=True, slots=True)
class PromptVersion:
    """Immutable snapshot of a prompt's content.

    Versions are never edited or removed; a change in content is a new version.
    Which version is in use is decided by the owning Prompt, not by the version.
    """

    number: int
    content: str
    created_at: datetime
    id: UUID = field(default_factory=uuid4)

    def __post_init__(self) -> None:
        if self.number < 1:
            raise InvalidPromptError("Version number must be >= 1")
        if not self.content.strip():
            raise InvalidPromptError("Version content cannot be blank")
```

Leia o trecho acima como uma frase: "*uma PromptVersion é um carimbo imutável com número, conteúdo, data e id; ao ser criada, recusa número menor que 1 e conteúdo em branco.*" É exatamente o que o código diz.

2.5 Protocol: contratos sem herança

Python usa a ideia de *duck typing*: "se anda como um pato e grasna como um pato, serve como pato". Um **Protocol** descreve a *forma* que um objeto precisa ter — quais métodos, com quais assinaturas — sem obrigar ninguém a herdar de nada. Qualquer classe que tenha esses métodos cumpre o contrato.

src/port/prompt_repository.py · um contrato

```python
class PromptRepository(Protocol):
    async def get(self, prompt_id: UUID) -> Prompt | None: ...

    async def save(self, prompt: Prompt) -> None:
        """Insert or update the whole aggregate, including all its versions."""
        ...

    async def list_active(self) -> Sequence[Prompt]:
        """Prompts whose status is ACTIVE."""
        ...

    async def list_deleted(self) -> Sequence[Prompt]:
        """Prompts whose status is DELETED (soft-deleted)."""
        ...
```

Os `...` no lugar do corpo significam "aqui só se declara; a implementação vem de outro lugar". Essa técnica é a base das **portas** que veremos no Capítulo 3.

2.6 async / await: o garçom que atende várias mesas

Boa parte do tempo de um serviço é **espera**: esperar o banco responder, esperar uma mensagem chegar pela rede. Em vez de ficar parado, o programa pode atender outra tarefa enquanto aguarda. Isso é **programação assíncrona**.

ANALOGIA · O garçom
Um garçom que serve uma mesa de cada vez, ficando plantado diante da cozinha até o prato sair, atende pouca gente. Um garçom assíncrono anota o pedido, vai à próxima mesa e volta quando a cozinha avisa. É um só garçom — mas ninguém espera à toa.

Palavra	Significado
`async def f(): ...`	Define uma função "que pode esperar" — uma **corrotina**.
`await algo()`	"Espero o resultado, mas enquanto isso o programa pode atender outras tarefas."
`asyncio.run(main())`	Abre o "restaurante": liga o motor que coordena todas as tarefas assíncronas.
`asyncio.Lock()`	Uma "chave única": só uma tarefa por vez pode segurá-la (veremos por que, no Capítulo 6).

NO PROJETO · Por que o projeto é assíncrono?
Porque suas duas fontes de espera — o NATS (rede) e o SQLite (disco) — têm bibliotecas assíncronas (*nats-py* e *aiosqlite*). Assim, o serviço atende várias mensagens sem travar.

2.7 Context managers: "with" e "async with"

O bloco `with` garante que algo seja **aberto e fechado direito**, aconteça o que acontecer no meio. Pense numa porta que se fecha sozinha ao sair. No projeto, a transação com o banco é assim:

Padrão que se repete nos casos de uso (resumido)

```
async with self._uow:
    prompt = await require_prompt(self._uow.prompts, command.prompt_id)
    ...
    await self._uow.commit()
```

Se algo der errado antes do `commit()`, ao sair do bloco a transação é **desfeita automaticamente**. Voltaremos a isto com calma.

2.8 Exceções: quando algo não pode continuar

`raise` interrompe a função e sinaliza um problema; `try / except` captura o sinal para tratá-lo. O projeto define **suas próprias exceções** para cada regra de negócio violada — por exemplo `PromptDeletedError` ("não dá para fazer isso num prompt apagado"). Nomes assim tornam o erro autoexplicativo e permitem tratá-lo de forma específica.

Mini-dicionário de símbolos

Símbolo	Significa
`_nome`	Convenção: "uso interno, não mexa de fora" (ex.: `self._status`).
`__init__`	O construtor: roda quando o objeto é criado.
`*, activate: bool = False`	O `*` obriga a escrever `activate=True` pelo nome; `= False` é o valor padrão.
`{...}`	Dicionário (pares chave: valor) ou conjunto, conforme o conteúdo.
`f"texto {x}"`	Texto com valores embutidos (*f-string*).

PAUSA PARA FIXAR · Teste o que aprendeu

1. Para que serve o `uv.lock` e por que não o editamos à mão?
2. O que significa `int | None` em `active_version_number: int | None`?
3. O que acontece em `async with uow:` se ocorrer um erro antes do `commit()`?

RESUMO DO CAPÍTULO

- **uv** cria o ambiente; `pyproject.toml` declara dependências; `uv.lock` fixa versões.
- Pastas com `__init__.py` são **pacotes**; `from x import y` traz coisas de outros módulos.
- **Tipos** (`str | None`) documentam e permitem checagem automática.
- **dataclass, enum, property, Protocol** dão forma aos dados e aos contratos.
- **async/await** evita esperas ociosas; **with** garante abrir/fechar corretamente; **exceções** próprias nomeiam os erros.

3 Arquitetura hexagonal

Por que o código tem cinco pastas e quem pode conversar com quem

Pergunta-guia: *por que o projeto não é "um monte de arquivos juntos", e que regra mantém a ordem?*

3.1 A ideia: proteger as regras da tecnologia

A **arquitetura hexagonal** (também chamada de *Ports and Adapters*, "portas e adaptadores", proposta por Alistair Cockburn) nasce de uma observação: **tecnologias mudam, regras de negócio duram**. Hoje usamos NATS e SQLite; amanhã pode ser HTTP e PostgreSQL. Se as regras ("só uma versão ativa", "apagar é soft") estiverem misturadas ao código do banco, cada troca de tecnologia vira uma cirurgia perigosa.

> **ANALOGIA · A tomada e o adaptador de viagem**
> Seu notebook (as regras) não sabe se a energia vem de uma hidrelétrica ou de um gerador. Ele só conhece o **formato do plugue** (a *porta*). Em cada país você usa um **adaptador** diferente, e o notebook continua intacto. No software: o núcleo define as portas; os adapters plugam as tecnologias.

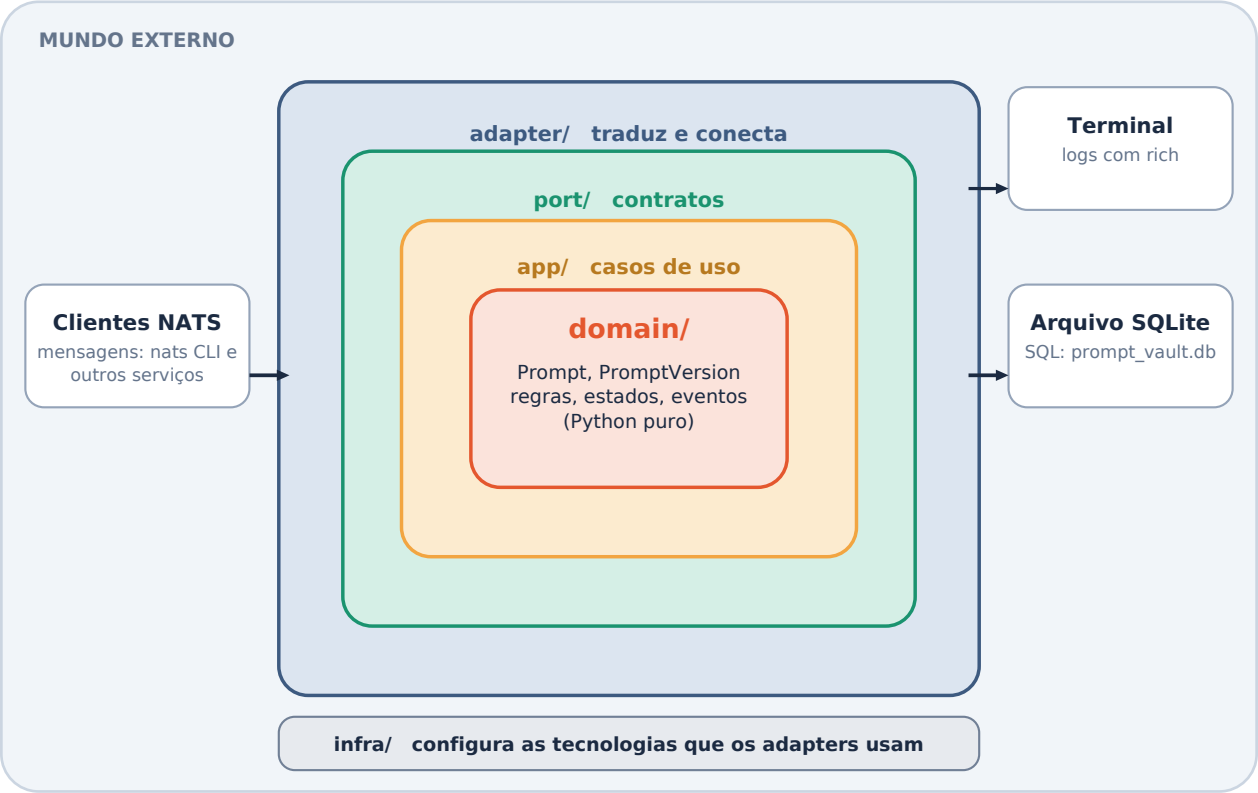


Figura 1 *— As camadas como anéis: o núcleo (domain) no centro, o mundo externo na borda. Tudo que entra ou sai atravessa os anéis.*

3.2 Os cinco blocos

Uma analogia ajuda a fixar o papel de cada pasta. Pense num **restaurante**:

Pasta	Papel	No restaurante	Exemplo no projeto
domain/	As regras do negócio, em Python puro	As receitas e as regras de higiene da cozinha	`Prompt`, `PromptStatus`, `VersionDiff`
app/	Casos de uso: coordena o fluxo	O chef que coordena o preparo de cada pedido	`CreateVersionUseCase`
port/	Contratos que o núcleo exige do mundo	O formulário de pedido e o contrato com fornecedores	`PromptRepository`, `UnitOfWork`
adapter/	Traduz e conecta tecnologias	O garçom (traduz o cliente) e o entregador (fornecedor)	`SqlAlchemyPromptRepository`, `MessagingAdapter`
infra/	Só configura as tecnologias	O equipamento: fogão, geladeira, telefone	`create_engine`, `NatsClient`

Há um sexto personagem, fora das pastas de camada: o `main.py`, a **raiz de composição**. É o único lugar que conhece todo mundo e "liga os fios": cria os adapters, entrega-os aos casos de uso e inicia o serviço (Capítulo 6).

3.3 A Regra de Dependência

Toda a disciplina da arquitetura cabe numa frase: **as dependências apontam sempre para dentro**. Os adapters conhecem a aplicação e o domínio; a aplicação conhece o domínio e as portas; o domínio **não conhece ninguém**. "Conhecer" aqui significa literalmente `import`.

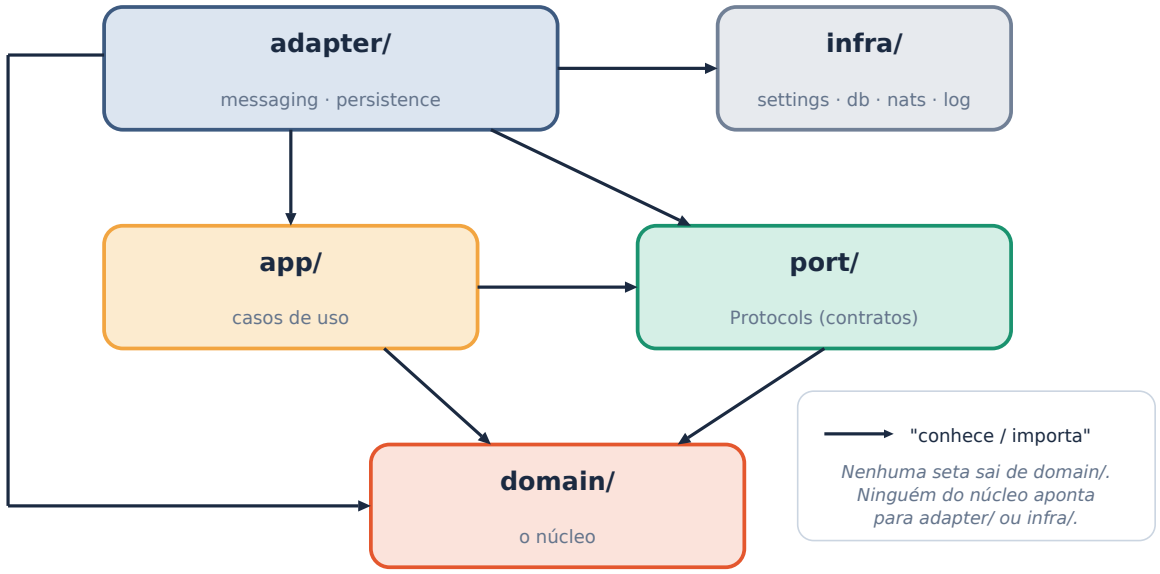


Figura 2 — A regra de dependência. As setas indicam quem importa quem. Repare: nenhuma seta nasce em domain/.

NO PROJETO · Esta regra é verificada, não apenas desejada

Durante o desenvolvimento, rodamos uma busca por qualquer menção a `sqlalchemy`, `nats`, `structlog`, `rich`, `adapter`, `infra` e até `logging` dentro de `domain/`, `app/` e `port/`. **O resultado foi sempre "nenhuma ocorrência".** E o verificador de tipos (pyright) terminou com 0 erros. O núcleo realmente não sabe que o NATS ou o SQLite existem.

3.4 Por que vale a pena (e o que custa)

Benefício	Como apareceu neste projeto
Testar sem tecnologia	Todo o domínio e todos os casos de uso foram testados com um repositório falso em memória — sem banco, sem NATS.
Trocar peças	Trocar SQLite por PostgreSQL mexeria só em `adapter/persistence` e na configuração de `infra/`.
Evoluir	Expor o sistema por HTTP seria um novo adapter ao lado do de mensageria; as regras não mudariam.
Entender	Dado um arquivo, você sabe a que camada pertence — e, portanto, o que ele pode (e não pode) fazer.

ATENÇÃO · O custo

Mais pastas e mais "cerimônia". Para um script de 100 linhas, seria exagero. Aqui vale pela natureza didática do projeto e porque ele foi desenhado para crescer: arquitetura é sempre uma troca consciente.

3.5 Os padrões que você vai encontrar

Arquiteturas costumam vir acompanhadas de "padrões de projeto": soluções nomeadas para problemas recorrentes. Aqui estão os do Prompt Vault:

Padrão	Em uma frase	Onde
Aggregate (DDD)	Um grupo de objetos tratado como unidade, com uma "raiz" que protege as regras.	`Prompt` (raiz) + `PromptVersion`
Repository	Esconde o armazenamento atrás de uma interface de "coleção".	`PromptRepository`
Unit of Work	Agrupa várias operações numa transação "tudo ou nada".	`UnitOfWork`
Strategy	Troca um algoritmo por outro sem mexer em quem o usa.	`DiffStrategy` / `LineDiffStrategy`
Factory	Uma função que fabrica objetos prontos para uso.	*factories* dos casos de uso em `main.py`
Dependency Injection	Quem usa uma peça a *recebe* pronta, em vez de criá-la.	construtores dos casos de uso
Command / Query	Separar o que altera do que apenas lê.	`app/commands` e `app/queries`
Domain Events	Registrar "fatos ocorridos" para quem quiser reagir.	`events.py` e `pull_events()`

PAUSA PARA FIXAR · Teste o que aprendeu

1. Por que o *domain* não pode importar `sqlalchemy`?
2. Qual(is) camada(s) você alteraria para trocar SQLite por PostgreSQL?
3. Qual a diferença entre uma **porta** e um **adapter**?

RESUMO DO CAPÍTULO

- A arquitetura hexagonal separa **regras** (duradouras) de **tecnologia** (trocável).
- Cinco blocos: **domain, app, port, adapter, infra** — mais o `main.py` que os liga.
- **Regra de ouro:** as dependências apontam para dentro; o domínio não conhece ninguém.
- Porta = contrato (Protocol). Adapter = implementação concreta desse contrato.

4 Domain: o coração

Onde moram as regras do negócio — e como elas se protegem sozinhas

Pergunta-guia: *onde ficam as regras do sistema e o que impede que alguém as quebre?*

A pasta `src/domain/` tem 605 linhas e uma propriedade rara: **nada nela sabe que existe um banco de dados, uma rede ou um terminal**. É Python puro. Isso a torna fácil de testar e de entender — é aqui que o negócio "mora".

4.1 Modelagem rica, não anêmica

Um modelo **anêmico** é feito só de dados; as regras ficam espalhadas por quem os usa. Um modelo **rico** faz o oposto: o próprio objeto *sabe se proteger*. Compare:

Ilustração da diferença (não é o código completo)

```
prompt.status = "deleted"      # anêmico: qualquer um muda, quando quiser, sem checar nada
prompt.delete()                # rico: o Prompt valida a transição, grava o horário e registra o evento
```

Todo o domínio do Prompt Vault segue a segunda abordagem: para mudar algo, você **pede** ao objeto, e ele decide.

4.2 O agregado: Prompt e suas versões

No jargão de *Domain-Driven Design*, um **agregado** é um conjunto de objetos tratados como uma unidade, com uma **raiz** que é o único ponto de entrada. Aqui, a raiz é o `Prompt`: as versões só nascem pelas mãos dele. É por isso que a numeração das versões (1, 2, 3…) **nunca** fica errada — ninguém mais pode criá-las.

Método do Prompt	O que faz	Evento registrado
`Prompt.create(name, content)`	Cria um prompt ativo, com a versão 1 ativa.	`PromptCreated`
`add_version(content, activate=False)`	Acrescenta a próxima versão; ativa só se pedido.	`PromptVersionAdded` (+ `PromptVersionActivated`)
`activate_version(n)`	Passa a servir a versão *n*. Se já é a ativa, não faz nada.	`PromptVersionActivated`
`deactivate()` / `reactivate()`	Desliga / religa o prompt.	`PromptDeactivated` / `PromptReactivated`
`delete()` / `restore()`	Soft delete e recuperação.	`PromptDeleted` / `PromptRestored`
`diff(a, b, strategy)`	Compara duas versões. Só lê.	nenhum
`Prompt.rehydrate(...)`	Reconstrói um prompt vindo do armazenamento, validando as regras.	nenhum

src/domain/entities/prompt.py · add_version

```python
def add_version(self, content: str, *, activate: bool = False) -> PromptVersion:
    """Append a new version. It becomes the active one only if `activate` is set."""
    self._ensure_not_deleted("add a version to")
    now = _utcnow()
    version = PromptVersion(number=len(self._versions) + 1, content=content, created_at=now)
    self._versions.append(version)
    self._touch(now)
    self._record(
        PromptVersionAdded(prompt_id=self._id, occurred_at=now, version_number=version.number)
    )
    if activate:
        self.activate_version(version.number)
    return version
```

Observe a ordem: **(1)** recusa se o prompt está apagado; **(2)** cria a versão (que valida o conteúdo por conta própria); **(3)** guarda; **(4)** atualiza a data; **(5)** registra o evento; **(6)** só então ativa, se foi pedido. Nenhuma dessas decisões vazou para fora do domínio.

4.3 Versões imutáveis e o ponteiro da versão ativa

Há duas ideias separadas, e confundi-las é o erro mais comum de iniciantes: as **versões** (conteúdo, imutável) e o **ponteiro** que diz qual delas está “valendo”. Mudar de versão ativa só move o ponteiro; ninguém reescreve o passado.



***Figura 3** — As versões nunca mudam; apenas o ponteiro de “ativa” se desloca. Voltar atrás (rollback) é só apontar para uma versão antiga.*

4.4 O ciclo de vida: uma máquina de estados

Um prompt está sempre em exatamente um de três estados — `ACTIVE`, `INACTIVE` ou `DELETED` — e só pode passar de um para outro por caminhos permitidos. Chamamos isso de **máquina de estados**. Ela está escrita de forma explícita, numa tabela de transições que qualquer pessoa consegue ler:

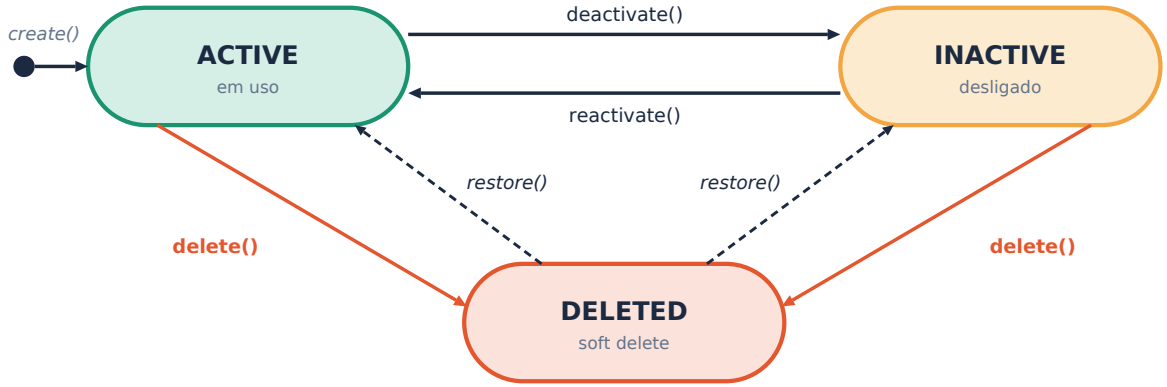


***Figura 4** — Os estados e as transições permitidas. Só restore() sai de DELETED, e volta ao estado em que o prompt estava antes de ser apagado.*

src/domain/status.py

```python
class PromptStatus(StrEnum):
    """Lifecycle state of a prompt."""

    ACTIVE = "active"
    INACTIVE = "inactive"
    DELETED = "deleted"

    def can_transition_to(self, target: "PromptStatus") -> bool:
        return target in _ALLOWED_TRANSITIONS[self]


# Explicit state machine. DELETED -> ACTIVE/INACTIVE is a restore (soft deletion).
_ALLOWED_TRANSITIONS: dict[PromptStatus, frozenset[PromptStatus]] = {
    PromptStatus.ACTIVE: frozenset({PromptStatus.INACTIVE, PromptStatus.DELETED}),
    PromptStatus.INACTIVE: frozenset({PromptStatus.ACTIVE, PromptStatus.DELETED}),
    PromptStatus.DELETED: frozenset({PromptStatus.ACTIVE, PromptStatus.INACTIVE}),
}
```

Apagar é **soft**: nenhuma linha é destruída. O prompt guarda `deleted_at` e o estado em que estava antes. Ao restaurar, ele volta exatamente para lá:

src/domain/entities/prompt.py · delete e restore

```python
def delete(self) -> None:
    """Soft delete: all versions are kept and the prompt can be restored."""
    previous = self._status
    now = self._transition_to(PromptStatus.DELETED)
    self._status_before_deletion = previous
    self._deleted_at = now
    self._record(PromptDeleted(prompt_id=self._id, occurred_at=now, previous_status=previous))

def restore(self) -> None:
    """Undo a soft delete, going back to the status the prompt had before it."""
    if not self.is_deleted:
        raise PromptNotDeletedError()
    target = self._status_before_deletion or PromptStatus.INACTIVE
    now = self._transition_to(target)
    self._status_before_deletion = None
    self._deleted_at = None
    self._record(PromptRestored(prompt_id=self._id, occurred_at=now, restored_status=target))
```

4.5 Invariantes: o que é sempre verdade

Uma **invariante** é uma afirmação que *nunca* pode ser falsa. O domínio garante todas estas; se alguém tentar violá-las, recebe uma exceção específica:

Invariante	Onde é garantida	Erro
Nome e conteúdo não podem ser vazios	`Prompt.__init__` e `PromptVersion.__post_init__`	`InvalidPromptError`
Versões numeradas 1, 2, 3… sem buracos	`add_version` e `Prompt.__init__`	`InvalidPromptError`
A versão ativa precisa existir	`activate_version` (via `get_version`)	`VersionNotFoundError`
Prompt apagado não recebe nem ativa versões	`_ensure_not_deleted`	`PromptDeletedError`
Só é possível restaurar o que está apagado	`restore`	`PromptNotDeletedError`
Só há transições de estado permitidas	`PromptStatus.can_transition_to`	`InvalidStateTransitionError`
`deleted_at` preenchido ⇔ estado DELETED	`Prompt.__init__`	`InvalidPromptError`
Comparar exige duas versões *distintas e existentes*	`Prompt.diff`	`InvalidVersionComparisonError`

4.6 Eventos de domínio: fatos ocorridos

Quando algo relevante acontece, o agregado **registra um evento**: um fato, no passado, imutável ("a versão 3 foi ativada"). Importante: o domínio apenas *registra*; quem decide o que fazer com os eventos (publicar no NATS, por exemplo) é uma camada externa, através do método `pull_events()`, que devolve e esvazia a lista.

src/domain/events.py

```python
@dataclass(frozen=True, slots=True, kw_only=True)
class DomainEvent:
    """Immutable fact about something that happened to a prompt."""

    prompt_id: UUID
    occurred_at: datetime


@dataclass(frozen=True, slots=True, kw_only=True)
class PromptCreated(DomainEvent):
    name: str
```

São sete eventos: `PromptCreated`, `PromptVersionAdded`, `PromptVersionActivated`, `PromptDeactivated`, `PromptReactivated`, `PromptDeleted`, `PromptRestored`. Eles deixam o sistema **pronto para uma arquitetura orientada a eventos**: outros serviços podem reagir a mudanças sem que o Prompt Vault saiba que eles existem.

4.7 O diff e o padrão Strategy

Comparar duas versões é uma regra do negócio, então mora no domínio. O resultado é um objeto **neutro** — nada de HTML, cores ou tela — composto de `DiffEntry` (cada linha, com o tipo da mudança: `ADDED`, `REMOVED`, `MODIFIED` ou `UNCHANGED`). Quem apresentar, que decida como mostrar.

O *algoritmo* de comparação, porém, é trocável. O `Prompt.diff` recebe uma **estratégia** — qualquer objeto com um `name` e um método `diff(old, new)` (um Protocol, lembra?):

src/domain/entities/prompt.py · diff

```python
def diff(self, from_number: int, to_number: int, strategy: DiffStrategy) -> VersionDiff:
    """Compare two distinct existing versions. Read-only: no state change, no events."""
    if from_number == to_number:
        raise InvalidVersionComparisonError(from_number)
    old = self.get_version(from_number)
    new = self.get_version(to_number)
    return VersionDiff(
        prompt_id=self._id,
        from_version=from_number,
        to_version=to_number,
        algorithm=strategy.name,
        entries=strategy.diff(old.content, new.content),
    )
```

A estratégia padrão, `LineDiffStrategy`, compara linha a linha usando o módulo `difflib` da biblioteca padrão do Python. O miolo dela é este laço, que traduz as "operações" do difflib em entradas do nosso modelo:

src/domain/diff/line_diff.py · o laço principal

```python
for tag, i1, i2, j1, j2 in matcher.get_opcodes():
    if tag == "equal":
        for offset in range(i2 - i1):
            entries.append(self._unchanged(old_lines, new_lines, i1 + offset, j1 + offset))
        continue

    # "replace" pairs lines up as modifications; leftovers are plain removals/additions.
    paired = min(i2 - i1, j2 - j1) if tag == "replace" else 0
    for offset in range(paired):
        entries.append(
            DiffEntry(
                ChangeType.MODIFIED,
                old_line=i1 + offset + 1,
                new_line=j1 + offset + 1,
                old_text=old_lines[i1 + offset],
                new_text=new_lines[j1 + offset],
            )
        )
    for i in range(i1 + paired, i2):
        entries.append(
            DiffEntry(ChangeType.REMOVED, old_line=i + 1, old_text=old_lines[i])
        )
    for j in range(j1 + paired, j2):
        entries.append(DiffEntry(ChangeType.ADDED, new_line=j + 1, new_text=new_lines[j]))
```

Um exemplo real, executado no sistema. Versão 1: `linha1 / linha2 / linha3 / linha4`. Versão 2: `linha1 / linha2 alterada / linha4 / linha5`. O diff devolvido foi:

Tipo	Linha antiga	Linha nova	Texto antigo	Texto novo
unchanged	1	1	linha1	linha1
modified	2	2	linha2	linha2 alterada
removed	3	—	linha3	—

Tipo	Linha antiga	Linha nova	Texto antigo	Texto novo
unchanged	4	3	linha4	linha4
added	—	4	—	linha5

NO PROJETO · Estendendo sem quebrar nada

Para criar uma comparação "palavra por palavra", basta escrever **uma classe nova** com `name` e `diff`, e entregá-la ao caso de uso no `main.py`. O domínio, o caso de uso `CompareVersions` e o adapter de mensageria continuam exatamente como estão. É esse o ganho do padrão Strategy.

4.8 Uma lição que o teste ensinou

LIÇÃO APRENDIDA · O bug que o smoke test encontrou

Na primeira versão do domínio, `restore()` **funcionava mesmo em prompts que não estavam apagados** — e, como não havia estado anterior guardado, ele *desativava* o prompt em silêncio. Pior: `deactivate()` e `reactivate()` aceitavam prompts apagados, deixando um estado contraditório (apagado mas "ativo") que o próprio domínio recusa ao carregar — o prompt seria gravado e depois ficaria impossível de ler.

O teste de ponta a ponta esperava um erro ao restaurar um prompt não apagado e **não o recebeu**. A correção foram três guardas curtas (o `PromptNotDeletedError` e o `_ensure_not_deleted` que você viu acima). Moral: regras pertencem ao domínio, e testar o sistema de verdade encontra buracos que a leitura do código não mostra.

PAUSA PARA FIXAR · Teste o que aprendeu

1. Por que `PromptVersion` é `frozen` (imutável)?
2. O que acontece se chamarmos `restore()` num prompt ativo?
3. O que você implementaria para comparar duas versões palavra por palavra?

RESUMO DO CAPÍTULO

- O **domain** é Python puro: regras, estados, eventos e o diff — sem tecnologia.
- **Prompt** é a raiz do agregado; **PromptVersion** é imutável; só o ponteiro da versão ativa se move.
- O ciclo de vida é uma **máquina de estados explícita**; apagar é *soft* e reversível.
- **Invariantes** são protegidas por exceções específicas; **eventos** registram fatos ocorridos.
- O diff usa o padrão **Strategy**: o algoritmo é trocável sem tocar no resto.

5 Ports e Application

Contratos e casos de uso: como um pedido vira ação sem depender de tecnologia

Pergunta-guia: *como o sistema executa uma ação (“criar versão”) sem saber onde os dados ficam nem por onde o pedido chegou?*

5.1 Ports: o que o núcleo exige do mundo

A pasta `port/` é minúscula (57 linhas) e importantíssima. Ela contém **três contratos**, todos escritos como *Protocols*. Cada um expressa uma necessidade: “preciso guardar e achar prompts”, “preciso avisar o mundo sobre eventos”, “preciso de uma transação”.

Porta	A necessidade	Métodos
`PromptRepository`	Guardar e recuperar prompts, como se fossem uma coleção	`get`, `save`, `list_active`, `list_deleted`
`EventPublisher`	Avisar o mundo sobre os eventos de domínio	`publish(events)`
`UnitOfWork`	Agrupar operações numa transação “tudo ou nada”	`prompts`, `commit`, `rollback`, `async with`

src/port/event_publisher.py e src/port/unit_of_work.py

```python
class EventPublisher(Protocol):
    async def publish(self, events: Sequence[DomainEvent]) -> None: ...

class UnitOfWork(Protocol):
    """Transactional boundary. Leaving the block without `commit()` must roll back."""

    prompts: PromptRepository

    async def __aenter__(self) -> Self: ...

    async def __aexit__(
        self,
        exc_type: type[BaseException] | None,
        exc: BaseException | None,
        tb: TracebackType | None,
    ) -> None: ...

    async def commit(self) -> None: ...

    async def rollback(self) -> None: ...
```

CONCEITO · Contratos mínimos
Uma porta só tem o que os casos de uso *realmente* precisam. Não existe `delete(id)` no repositório porque apagar é uma regra do domínio (`prompt.delete()`), seguida de `save`. O repositório não decide nada: só guarda e busca.

5.2 Unit of Work: a transação “tudo ou nada”

> **ANALOGIA · A transferência bancária**
> Transferir R$ 100 da conta A para a B são duas operações: debitar A e creditar B. Se a luz cair entre as duas, o dinheiro sumiria. Uma **transação** garante que ou *as duas* acontecem, ou *nenhuma*. O Unit of Work é o objeto que abre essa “caixa protegida”, confirma (`commit`) ou desfaz (`rollback`) tudo.

No projeto: entrou no `async with uow:`, tudo que for gravado só vale se chegar ao `commit()`. Se uma exceção ocorrer antes, ao sair do bloco acontece o *rollback* e **nada** fica gravado.

5.3 Casos de uso: a orquestra

Um **caso de uso** é uma ação que o sistema oferece, do ponto de vista de quem o usa: “criar um prompt”, “ativar uma versão”. A camada `app/` tem 311 linhas e contém exatamente onze casos de uso, divididos em dois grupos clássicos:

- **Commands** (escrita): *alteram* o estado. Usam o Unit of Work e publicam eventos.
- **Queries** (leitura): só *consultam*. Não abrem transação nem publicam nada.

Caso de uso	Tipo	Orquestra	Devolve
`CreatePrompt`	command	`Prompt.create`	id do prompt
`CreateVersion`	command	`prompt.add_version(activate=…)`	nº da versão
`UpdateContent`	command	`prompt.add_version(activate=True)`	nº da versão
`ActivateVersion`	command	`prompt.activate_version(n)`	—
`DeletePrompt`	command	`prompt.delete()`	—
`RestorePrompt`	command	`prompt.restore()`	—
`GetPrompt`	query	repositório → `get`	o Prompt
`ListVersions`	query	`prompt.versions`	as versões
`ListActivePrompts`	query	`list_active`	lista de prompts
`ListDeletedPrompts`	query	`list_deleted`	lista de prompts
`CompareVersions`	query	`prompt.diff(a, b, strategy)`	o `VersionDiff`

5.4 Anatomia de um caso de uso

src/app/commands/create_version.py

```python
class CreateVersionUseCase:
    def __init__(self, uow: UnitOfWork, publisher: EventPublisher) -> None:
        self._uow = uow
        self._publisher = publisher

    async def execute(self, command: CreateVersionCommand) -> int:
        """Add a new version to an existing prompt. Returns the new version number."""
        async with self._uow:
            prompt = await require_prompt(self._uow.prompts, command.prompt_id)
            version = prompt.add_version(command.content, activate=command.activate)
            await self._uow.prompts.save(prompt)
            await self._uow.commit()
        await self._publisher.publish(prompt.pull_events())
        return version.number
```

1. O **construtor recebe** o Unit of Work e o publicador. O caso de uso não cria nada: só usa o que lhe entregaram.
2. `async with self._uow` abre a transação.
3. `require_prompt` carrega o prompt (ou lança `PromptNotFoundError`).
4. `prompt.add_version(...)`: a regra de negócio é chamada *no domínio*. O caso de uso não decide nada.
5. `save` + `commit`: persiste e confirma.
6. Só **depois do commit** os eventos são publicados e o número da versão é devolvido.

ATENÇÃO · Por que publicar depois do commit?
Se publicássemos antes, um erro na gravação deixaria o mundo sabendo de algo que não aconteceu. O custo: se a publicação falhar *depois* do commit, o dado já está salvo mas o evento se perde. Uma solução madura para isso é o padrão *outbox* (gravar o evento junto, na mesma transação) — ainda não implementado aqui, e registrado como próximo passo.

5.5 Injeção de dependência e o poder do “falso”

Como o caso de uso **recebe** suas dependências (em vez de criá-las), podemos entregar qualquer objeto que cumpra o contrato — inclusive um **falso**, feito só para testar. Foi assim que verificamos todos os casos de uso sem banco e sem NATS. Este é o repositório falso usado nos testes:

Repositório falso (script de verificação, fora do projeto)

```
class FakeRepo:
    def __init__(self): self.store = {}
    async def get(self, i): return self.store.get(i)
    async def save(self, p): self.store[p.id] = p; log.append("save")
    async def list_active(self): return [p for p in self.store.values() if p.status is PromptStatus.ACTIVE]
    async def list_deleted(self): return [p for p in self.store.values() if p.is_deleted]
```

Seis linhas bastaram para testar a aplicação inteira. Esse é o prêmio da arquitetura: **a lógica pode ser exercitada isoladamente**.

NO PROJETO · Erros da aplicação
O domínio não sabe “buscar” prompts — ele só os recebe. Por isso quem lança `PromptNotFoundError` (“esse id não existe”) é a camada `app`, em `require_prompt`. Os erros de *regra de negócio* (`DomainError` e filhos) atravessam a aplicação sem tradução; o adapter de mensageria os converterá em códigos de erro.

PAUSA PARA FIXAR · Teste o que aprendeu

1. Qual a diferença entre um **command** e uma **query**?
2. Por que os eventos só são publicados *depois* do commit?
3. Por que o caso de uso recebe o `uow` no construtor em vez de criá-lo?

RESUMO DO CAPÍTULO

- **Ports** são três contratos mínimos: repositório, publicador de eventos e unit of work.
- **Casos de uso** só coordenam: carregam, chamam o domínio, salvam e publicam.
- **Commands** escrevem (com transação e eventos); **queries** só leem.
- **Injeção de dependência** torna possível testar com objetos falsos.

6 Adapters e Infra

Como o núcleo se conecta ao NATS, ao SQLite e ao terminal

Pergunta-guia: *como um núcleo que "não conhece tecnologia nenhuma" consegue guardar dados e receber mensagens?*

Aqui entram as 983 linhas que ligam o núcleo ao mundo real: **833** nos adapters e **150** na infra. Uma regra de ouro vale para elas: **nenhuma regra de negócio**. Adapters apenas *traduzem* e *conectam*; a infra apenas *configura*.

6.1 Infra: configurando as tecnologias

A pasta `infra/` contém quatro arquivos, cada um configurando uma tecnologia. Ela **não importa nada do projeto** — é só ferramental.

Arquivo	Responsabilidade
`settings.py`	Lê as configurações de variáveis de ambiente, com valores padrão.
`database.py`	Cria o motor do SQLite, aplica os ajustes de concorrência e cria as tabelas.
`nats_client.py`	Abre e fecha a conexão com o NATS e registra os eventos de conexão.
`logging.py`	Configura o structlog e o rich — uma vez, em um só lugar.

Variável de ambiente	Padrão	Para que serve
`NATS_URL`	`nats://localhost:4222`	Endereço do servidor NATS.
`DATABASE_URL`	`sqlite+aiosqlite:///./prompt_vault.db`	Onde fica o banco (um arquivo).
`LOG_LEVEL`	`INFO`	Quanto detalhe aparece nos logs.
`HANDLER_TIMEOUT_SECONDS`	`5`	Tempo máximo para processar uma mensagem.

6.2 Persistência: SQLAlchemy e SQLite

SQLite e o modo WAL

O **SQLite** é um banco de dados que cabe num *arquivo*: não há servidor para instalar. Para uso concorrente, ajustamos algumas opções toda vez que uma conexão é aberta:

src/infra/database.py

```python
if engine.url.get_backend_name() == "sqlite":

    @event.listens_for(engine.sync_engine, "connect")
    def _configure_sqlite(dbapi_connection, _record) -> None:
        cursor = dbapi_connection.cursor()
        cursor.execute("PRAGMA journal_mode=WAL")  # readers don't block the writer
        cursor.execute("PRAGMA synchronous=NORMAL")  # safe with WAL, much faster
        cursor.execute("PRAGMA foreign_keys=ON")
        cursor.execute("PRAGMA busy_timeout=5000")  # wait for locks instead of failing
        cursor.close()
```

ANALOGIA · O caderno com rascunho (WAL)

No modo comum, quando alguém escreve no “caderno”, ninguém pode lê-lo. No modo **WAL** (*write-ahead log*), as novas anotações vão primeiro para um rascunho ao lado; os leitores continuam lendo o caderno enquanto o escritor anota. Depois, o rascunho é incorporado. Resultado: leituras e escrita convivem. Os outros ajustes: `foreign_keys=ON` faz o banco exigir que as relações entre tabelas sejam válidas, e `busy_timeout` manda esperar um pouco por um bloqueio em vez de falhar na hora.

O ORM: tabelas como classes

Um **ORM** (*Object-Relational Mapper*) como o SQLAlchemy permite descrever uma tabela como uma classe Python. Estas classes **não são** as do domínio: são o *modelo relacional*, e nunca saem do adapter.

src/adapter/persistence/models.py

```python
class PromptModel(Base):
    __tablename__ = "prompts"

    id: Mapped[str] = mapped_column(String(36), primary_key=True)
    name: Mapped[str] = mapped_column(Text)
    status: Mapped[str] = mapped_column(String(16), index=True)
    active_version_number: Mapped[int | None] = mapped_column(Integer, nullable=True)
    status_before_deletion: Mapped[str | None] = mapped_column(String(16), nullable=True)
    created_at: Mapped[datetime] = mapped_column(UTCDateTime)
    updated_at: Mapped[datetime] = mapped_column(UTCDateTime)
    deleted_at: Mapped[datetime | None] = mapped_column(UTCDateTime, nullable=True)
```

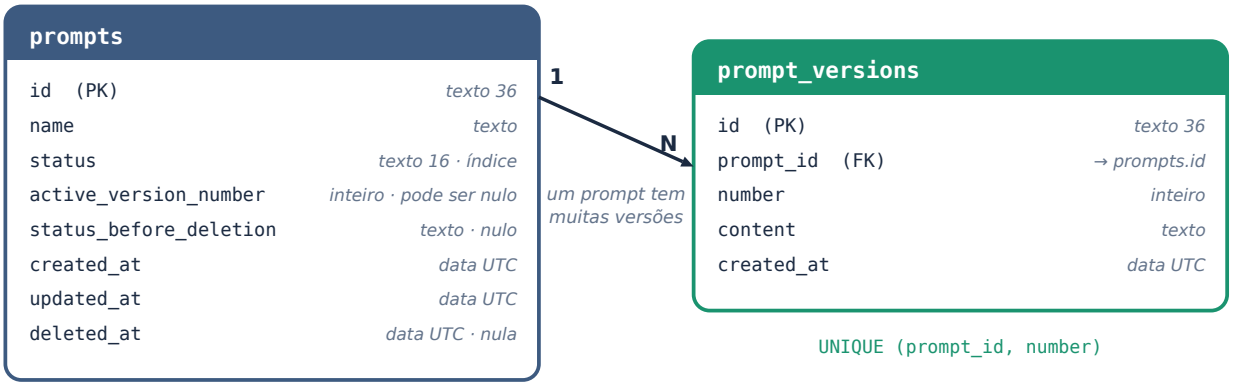


***Figura 5** — As duas tabelas. Um prompt tem muitas versões; a restrição UNIQUE impede dois números iguais no mesmo prompt.*

Domínio ≠ tabela: o mapper

Por que dois modelos? Porque servem a donos diferentes. O domínio muda por razões de *negócio*; as tabelas, por razões de *armazenamento*. O **mapper** faz a tradução nas duas direções. Ao ler, ele reconstrói o agregado com `Prompt.rehydrate`, que revalida as regras e **não emite eventos** (reconstituir não é um fato novo):

src/adapter/persistence/mapper.py

```
def to_domain(row: PromptModel) -> Prompt:
    versions = [
        PromptVersion(
            id=UUID(version.id),
            number=version.number,
            content=version.content,
            created_at=version.created_at,
        )
        for version in row.versions
    ]
    status_before_deletion = row.status_before_deletion
    return Prompt.rehydrate(
```

Ao gravar, o repositório respeita a natureza do histórico — **só acrescenta** versões:

src/adapter/persistence/prompt_repository.py

```
async def save(self, prompt: Prompt) -> None:
    row = await self._fetch_row(str(prompt.id))
    if row is None:
        self._session.add(to_new_row(prompt))
    else:
        apply_state(row, prompt)
        stored_numbers = {version.number for version in row.versions}
        # History is append-only: only versions that are not stored yet are inserted.
        for version in prompt.versions:
            if version.number not in stored_numbers:
                row.versions.append(version_to_row(prompt.id, version))
    await self._session.flush()
    log.info("prompt_saved", prompt_id=str(prompt.id), versions=len(prompt.versions))
```

NO PROJETO · Um detalhe de fuso horário

O SQLite não guarda fuso horário: ele devolveria datas “soltas”. Um tipo próprio, `UTCDateTime`, grava tudo em UTC e devolve datas *com fuso*. Um teste confirmou: as datas voltam do banco idênticas ao que o domínio criou.

O Unit of Work concreto e a chave de escrita

src/adapter/persistence/unit_of_work.py

```python
async def __aenter__(self) -> Self:
    await self._write_lock.acquire()
    try:
        self._session = self._session_factory()
        self.prompts = SqlAlchemyPromptRepository(self._session)
        self._committed = False
    except BaseException:
        self._write_lock.release()
        raise
    return self

async def __aexit__(
    self,
    exc_type: type[BaseException] | None,
    exc: BaseException | None,
    tb: TracebackType | None,
) -> None:
    session = self._require_session()
    try:
        if not self._committed:
            await session.rollback()
            log.info("transaction_rolled_back", failed=exc_type is not None)
    finally:
        await session.close()
        self._session = None
        self._write_lock.release()
```

ANALOGIA · Duas pessoas, uma planilha

Se duas pessoas abrem a mesma planilha, cada uma altera uma célula e salva, a que salvar por último apaga a mudança da outra — a chamada **atualização perdida**. O `asyncio.Lock` em `__aenter__` é a "chave única da sala": enquanto uma transação de escrita está aberta, as outras esperam na fila. Testamos com 40 comandos simultâneos sobre o mesmo prompt: as versões saíram de 2 a 41, **sem perda e sem duplicata**.

ATENÇÃO · Uma concessão honesta

As queries não precisam de transação, então usam um repositório que abre uma sessão curta a cada chamada (`SessionPerCallPromptRepository`). Como a porta exige `save`, ele existe mas lança `NotImplementedError`. A solução "limpa" seria uma porta só de leitura (`PromptReader`), mexendo em `port/` e nas queries. Ficou como melhoria futura.

6.3 Mensageria: o adapter do NATS

O **NATS** é um sistema de mensagens: um servidor central (o *broker*) onde clientes **publicam** mensagens em *assuntos* (*subjects*) e outros **assinam** esses assuntos. Os nomes usam pontos para formar hierarquias (`prompt.events.created`) e o curinga `>` significa "tudo abaixo" (`prompt.events.>` pega todos os eventos). Há três padrões de uso, todos presentes aqui:



Figura 6 — Os três padrões de conversa no Prompt Vault.

Os subjects do sistema

Tipo	Subjects	Modo
Commands	`prompt.create`, `prompt.version.create`, `prompt.content.update`, `prompt.version.activate`, `prompt.delete`, `prompt.restore`	Assíncronos. Respondem *se* o remetente informar um endereço de resposta.
Queries	`prompt.get`, `prompt.versions.list`, `prompt.list.active`, `prompt.list.deleted`, `prompt.versions.compare`	Pedido e resposta. Sem endereço de resposta, são ignoradas.
Eventos (saída)	`prompt.events.created`, `…version_added`, `…version_activated`, `…deleted`, `…restored`, `…deactivated`, `…reactivated`	Publicar e assinar.
Erros (saída)	`prompt.errors`	Erros de commands que não tinham a quem responder.

O correlation id: o código de rastreio

ANALOGIA · O código de rastreio dos Correios

Com um código de rastreio, você segue a encomenda por todas as etapas. O **correlation id** faz o mesmo com uma mensagem: ele entra pelo cabeçalho `X-Correlation-Id` (ou o sistema cria um) e **acompanha** a resposta, os eventos publicados e *cada linha de log*. Depois, basta buscar esse código para reconstruir a história completa.

src/adapter/messaging/correlation.py

```python
@contextmanager
def correlation_scope(correlation_id: str) -> Iterator[None]:
    """Make the id available to the publisher and to every log line inside the block."""
    token = _current.set(correlation_id)
    structlog.contextvars.bind_contextvars(correlation_id=correlation_id)
    try:
        yield
    finally:
        structlog.contextvars.unbind_contextvars("correlation_id")
        _current.reset(token)
```

Ele usa uma *ContextVar*: uma variável cujo valor é “privado” de cada tarefa assíncrona. Assim, mensagens simultâneas não misturam seus ids.

O adapter só traduz

Veja o "handler" de um command. Ele valida o *formato* (é um UUID? é um texto?), monta o comando e chama o caso de uso. **Nada de regra de negócio**:

src/adapter/messaging/handlers.py

```
async def create_version(p: dict[str, Any]) -> Any:
    command = CreateVersionCommand(
        prompt_id=require_uuid(p, "prompt_id"),
        content=require_str(p, "content"),
        activate=optional_bool(p, "activate"),
    )
    return {"version_number": await factories.create_version().execute(command)}
```

Já o coração do adapter decide o que fazer com sucessos, falhas e demora. Leia com atenção o `asyncio.wait_for` (o **timeout**) e a separação entre erros "esperados" (*warning*) e "inesperados" (*exception*, com traceback):

src/adapter/messaging/server.py

```
async def _execute(
    self,
    route: Route,
    payload: dict[str, Any],
    decode_error: InvalidPayloadError | None,
    correlation_id: str,
) -> tuple[bool, bytes]:
    try:
        if decode_error is not None:
            raise decode_error
        log.info("use_case_started", use_case=route.use_case)
        data = await asyncio.wait_for(route.handler(payload), timeout=self._timeout)
        log.info("use_case_completed", use_case=route.use_case)
        return True, ok_envelope(correlation_id, data)
    except Exception as exc:
        code, message, unexpected = to_error(exc)
        if unexpected:
            log.exception("use_case_failed", use_case=route.use_case, error_code=code)
        else:
            log.warning("use_case_rejected", use_case=route.use_case, error_code=code, error=message)
        return False, error_envelope(correlation_id, code, message, subject=route.subject)
```

O envelope e os códigos de erro

Toda resposta é um envelope `{ok, correlation_id, data | error}`. Um erro real, produzido ao forçarmos um timeout:

Resposta real de erro

```
{"ok": false, "correlation_id": "ae85212c-327d-4d49-a689-64493a259c1f",
 "error": {"code": "timeout", "message": "The operation timed out"}, "subject": "prompt.list.active"}
```

Código	Quando acontece	Exemplo
`invalid_payload`	Mensagem malformada: JSON inválido, campo ausente ou de tipo errado, UUID mal formado.	`{not json`
`not_found`	O prompt pedido não existe.	um UUID aleatório
`domain_error`	Uma regra de negócio foi violada.	ativar a versão 99; restaurar prompt ativo
`timeout`	O processamento passou do limite.	handler lento
`internal_error`	Erro inesperado. **Detalhes só no log**, nunca na resposta.	falha imprevista

Publicando os eventos

src/adapter/messaging/event_publisher.py

```python
async def publish(self, events: Sequence[DomainEvent]) -> None:
    correlation_id = get_correlation_id()
    headers = {CORRELATION_HEADER: correlation_id} if correlation_id else None
    for event in events:
        name = _EVENT_NAMES.get(type(event), type(event).__name__.lower())
        subject = f"{subjects.EVENTS_PREFIX}.{name}"
        body: dict[str, Any] = {"event": name, **dataclasses.asdict(event)}
        try:
            await self._nats_client.nc.publish(subject, encode(body), headers=headers)
            log.info("event_published", subject=subject, prompt_id=str(event.prompt_id))
        except Exception:
            log.exception("event_publish_failed", subject=subject)
```

Repare no `try / except`: se a publicação falhar, o erro é **registrado, mas não propagado**. O motivo: o dado já foi gravado; dizer ao cliente que o comando falhou faria ele tentar de novo e duplicar o efeito.

6.4 Logging estruturado: structlog + rich

Um log tradicional é uma frase solta: “usuário 42 apagou o prompt”. Para achar algo, você precisa adivinhar o texto. Um log **estruturado** separa o *evento* dos seus *campos* (`event=use_case_started use_case=DeletePrompt correlation_id=fail-1`), e cada campo vira algo pesquisável. O **structlog** produz isso; o **rich** cuida da apresentação no terminal. O `correlation_id` é adicionado *automaticamente* a toda linha, graças à ContextVar que vimos.

Este é o rastro real de um command que **falhou** (apagar um prompt inexistente, sem endereço de resposta), condensado para caber na página:

Log real (condensado)

```
INFO     event_received           subject=prompt.delete kind=command has_reply=False correlation_id=fail-1
INFO     use_case_started         use_case=DeletePrompt                      correlation_id=fail-1
INFO     transaction_rolled_back  failed=True                                correlation_id=fail-1
WARNING  use_case_rejected        use_case=DeletePrompt error_code=not_found  correlation_id=fail-1
INFO     error_published          subject=prompt.errors source=prompt.delete  correlation_id=fail-1
```

Leia de cima para baixo: a mensagem chegou, o caso de uso começou, a transação foi *desfeita* (nada gravado), o erro foi classificado como `not_found` e publicado em `prompt.errors` — tudo com o mesmo `correlation_id`. **O domínio não importa o logger**: os logs vivem só nos adapters.

LIÇÃO APRENDIDA · Quando o rich atrapalha
Ao testar o timeout, a linha de erro saiu como um dicionário bruto, em vez do formato normal. A causa: o *RichHandler*, com tracebacks ligados, ignora o formatador do structlog quando há exceção. A solução foi desligar essa opção e deixar o próprio structlog formatar o traceback (também com rich). Moral: integrar duas bibliotecas boas exige testar a *junção* — inclusive no caminho de erro.

6.5 O main.py: a raiz de composição

Tudo isso são peças soltas até que alguém as ligue. Esse alguém é o `main.py`, a **raiz de composição**: o único arquivo que conhece todas as camadas.

#	Passo	Por quê
1	Configurar o logging e ler as *settings*	Antes de tudo, para registrar o que vier depois.

#	Passo	Por quê
2	Criar o motor SQLite e as tabelas	O banco precisa existir antes da primeira mensagem.
3	Conectar ao NATS	Se falhar, o programa registra o erro e encerra.
4	Criar o publicador, a chave de escrita e o repositório de leitura	Peças compartilhadas por todas as mensagens.
5	Montar as *factories* dos casos de uso	Uma instância **nova** por mensagem (veja abaixo).
6	Iniciar o MessagingAdapter (assinar os subjects)	A partir daqui o serviço atende.
7	Esperar `SIGINT`/`SIGTERM` e encerrar limpo	Cancelar assinaturas, drenar o NATS, fechar o banco.

src/main.py · as factories

```
def uow() -> SqlAlchemyUnitOfWork:
    return SqlAlchemyUnitOfWork(session_factory, write_lock)

factories = UseCaseFactories(
    create_prompt=lambda: CreatePromptUseCase(uow(), publisher),
    create_version=lambda: CreateVersionUseCase(uow(), publisher),
    update_content=lambda: UpdateContentUseCase(uow(), publisher),
    activate_version=lambda: ActivateVersionUseCase(uow(), publisher),
    delete_prompt=lambda: DeletePromptUseCase(uow(), publisher),
    restore_prompt=lambda: RestorePromptUseCase(uow(), publisher),
    get_prompt=lambda: GetPromptUseCase(reader),
    list_versions=lambda: ListVersionsUseCase(reader),
    list_active_prompts=lambda: ListActivePromptsUseCase(reader),
    list_deleted_prompts=lambda: ListDeletedPromptsUseCase(reader),
    compare_versions=lambda: CompareVersionsUseCase(reader, diff_strategy),
)
```

CONCEITO · Por que “uma instância por mensagem”?

Cada caso de uso de escrita carrega o seu Unit of Work — e, portanto, a sua *sessão* de banco. Uma sessão não pode ser compartilhada por mensagens simultâneas. Por isso o `lambda` (uma função anônima) fabrica um caso de uso novo, com sessão própria, a cada mensagem recebida.

PAUSA PARA FIXAR · Teste o que aprendeu

1. Para que serve o *correlation id*?
2. Por que o app cria uma instância nova de cada caso de uso para cada mensagem?
3. O que o modo WAL do SQLite melhora?

RESUMO DO CAPÍTULO

- **infra** só configura; **adapters** só traduzem e conectam — nenhuma regra de negócio.
- **Persistência**: modelo relacional separado do domínio, com mapper; histórico só cresce; um lock evita atualizações perdidas.
- **Mensageria**: subjects para commands e queries, envelope padronizado, códigos de erro, timeout e correlation id.
- **Logs estruturados** com structlog + rich; o correlation id costura tudo.
- O **main.py** liga as peças — é o único que conhece todas as camadas.

7 Uma mensagem de ponta a ponta

O que acontece, passo a passo, quando alguém altera um prompt

Pergunta-guia: *quando eu mando uma mensagem, por quais peças ela passa — e em que ordem?*

Agora que você conhece cada camada, vamos costurar tudo seguindo **uma única mensagem**: alguém altera o conteúdo de um prompt. Esta é a mensagem enviada ao subject `prompt.content.update`:

O pedido

```
nats request prompt.content.update \
  '{"prompt_id": "<ID>", "content": "E ai\nmundo\nfim"}' \
  -H "X-Correlation-Id:meu-rastreio"
```

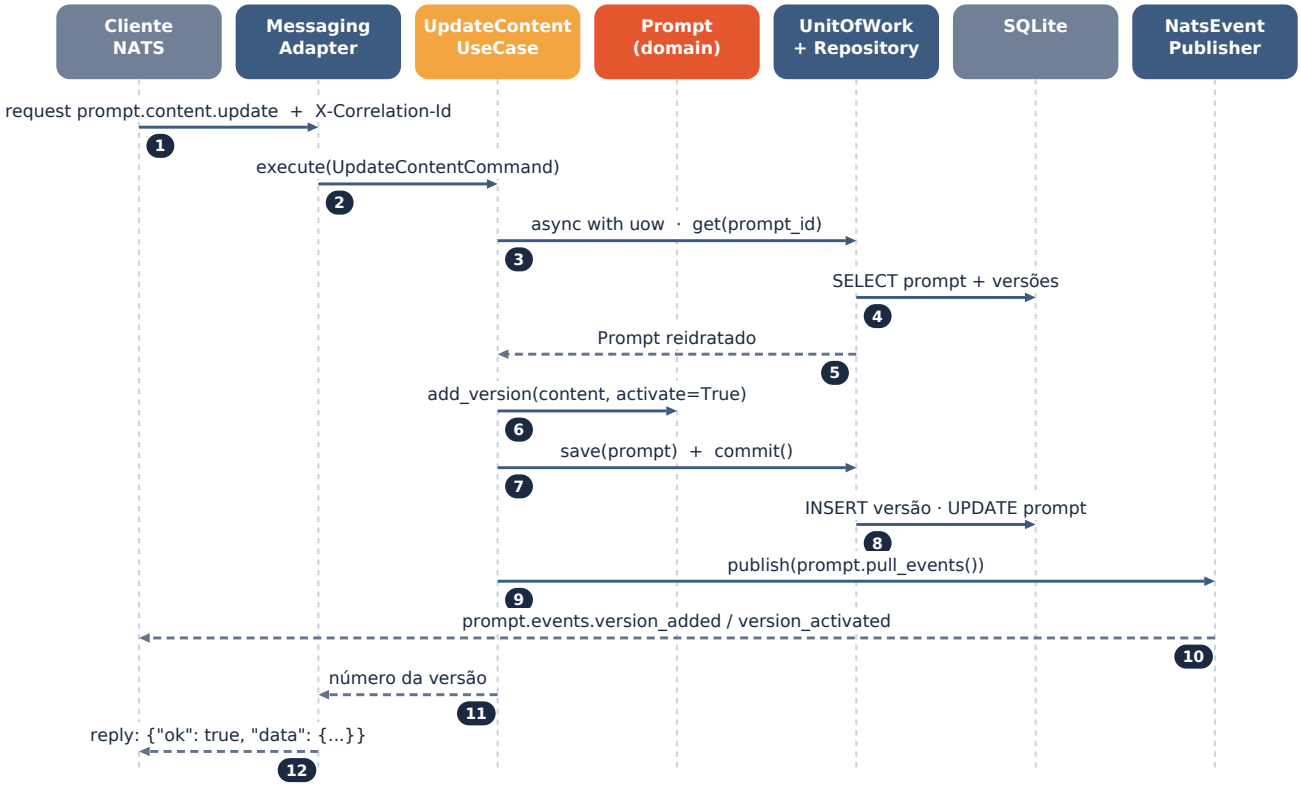


***Figura 7** — O caminho da mensagem. As linhas pontilhadas são retornos ou publicações; os números das setas aparecem na coluna "Setas" da tabela abaixo.*

7.1 Passo a passo

#	O que acontece	Setas	Quem faz	Camada
1	A mensagem chega ao subject `prompt.content.update`. O adapter decodifica o JSON, descobre o correlation id (do cabeçalho) e o registra em todos os logs.	1	`MessagingAdapter`	adapter

#	O que acontece	Setas	Quem faz	Camada
2	O handler valida o *formato* (UUID e texto), monta o `UpdateContentCommand` e pede à *factory* um caso de uso novo.	2	`handlers.py`	adapter
3	O caso de uso abre a transação (`async with uow`): o Unit of Work pega a chave de escrita e abre uma sessão.	3	`UpdateContentUseCase`	app
4	O repositório faz o `SELECT` do prompt e das versões; o mapper reconstrói o agregado com `rehydrate`.	4–5	`SqlAlchemyPromptRepository`	adapter
5	O caso de uso chama `prompt.add_version(content, activate=True)`. **A regra de negócio roda aqui**: valida, numera, ativa e registra dois eventos.	6	`Prompt`	domain
6	O repositório grava (`INSERT` da versão nova, `UPDATE` do prompt) e o Unit of Work faz o `commit`. A chave é liberada.	7–8	`SqlAlchemyUnitOfWork`	adapter
7	Já fora da transação, o caso de uso entrega os eventos (`pull_events()`) ao publicador, que os envia ao NATS com o correlation id.	9–10	`NatsEventPublisher`	adapter
8	O caso de uso devolve o número da versão; o adapter monta o envelope `ok` e o publica no endereço de resposta do cliente.	11–12	`MessagingAdapter`	adapter

7.2 O que cada observador enxerga

Quem	O que vê
O **cliente** (quem fez o request)	Uma resposta `{"ok": true, "correlation_id": "meu-rastreio", "data": {"version_number": N}}`.
Um **assinante** de `prompt.events.>`	Dois eventos — `version_added` e `version_activated` — ambos com o cabeçalho `X-Correlation-Id: meu-rastreio`.
O **banco**	Uma linha nova em `prompt_versions` e o `active_version_number` atualizado em `prompts`. As versões antigas permanecem intactas.
Os **logs**	Linhas `event_received`, `use_case_started`, `prompt_saved`, `transaction_committed`, `event_published`, `use_case_completed`, `reply_sent` — todas com `correlation_id=meu-rastreio`.

RESUMO DO CAPÍTULO

- Uma mensagem atravessa: **adapter** (entrada) → **app** (orquestra) → **domain** (decide) → **adapter** (grava e publica) → **adapter** (responde).
- O correlation id acompanha a mensagem do início ao fim: resposta, eventos e logs.
- **A lição de arquitetura:** o NATS só aparece nos passos 1, 2, 7 e 8; o SQL, nos passos 4 e 6. O passo 5 — a regra de negócio — não toca em nenhuma tecnologia. É isso ter o negócio no centro e a tecnologia nas bordas.

8 Verificando que funciona

Testes de cima a baixo — e as lições que eles deram

Pergunta-guia: *como sabemos que tudo isso realmente funciona, e não apenas parece funcionar?*

8.1 Verificar em camadas

Cada etapa do projeto foi verificada *no nível em que fazia sentido*. Quanto mais para dentro, mais rápido e isolado é o teste; quanto mais para fora, mais próximo da realidade:

Nível	O que foi testado	Com o quê
Domínio	Criação, versões, ativação, ciclo de vida, soft delete, eventos, reidratação, diff.	Python puro, sem nada externo
Aplicação	Os 11 casos de uso, incluindo "nada é salvo nem publicado quando o domínio recusa".	Repositório e publicador falsos
Persistência	Ida e volta ao SQLite, fuso horário, rollback, 25 comandos simultâneos, modo WAL, dados após reiniciar.	SQLite real em arquivo temporário
Ponta a ponta	Todos os subjects, eventos, erros, timeout, desligamento limpo.	NATS real (Docker) + app real
Smoke test	43 verificações pela "porta da frente", só com o NATS CLI.	O script `smoke_test.sh`

8.2 O smoke test

Smoke test ("teste de fumaça") vem da eletrônica: liga-se o aparelho e vê-se se sai fumaça. Em software, é uma verificação rápida de que o essencial funciona. O `smoke_test.sh` é um script de shell que usa **apenas o NATS CLI** — exatamente como um usuário real —, envia mensagens e confere as respostas. Duas peças pequenas sustentam o script:

smoke_test.sh · enviar um pedido e comparar resultados

```
req() {
  local subject=$1 payload=$2 cid=${3:-}
  local args=(request "$subject" "$payload" --timeout "$TIMEOUT" --raw)
  [[ -n "$cid" ]] && args+=(-H "X-Correlation-Id:$cid")
  RESP="$(nats "${args[@]}" 2>/dev/null)" || RESP="<sem resposta em $TIMEOUT>"
}

expect_eq() { # <descrição> <esperado> <obtido>
  if [[ "$3" == "$2" ]]; then pass "$1"; else fail "$1" "esperado: $2 | obtido: $3"; fi
}
```

Resultado da última execução contra a aplicação em funcionamento:

Grupo de verificações	Itens	Resultado
Commands (create, version.create, content.update, get, activate)	11	**11 / 11**
Queries (versions.list, compare, list.active)	6	**6 / 6**

Grupo de verificações	Itens	Resultado
Soft delete e recuperação	6	**6 / 6**
Erros (invalid_payload, not_found, domain_error)	8	**8 / 8**
Eventos em `prompt.events.>`, com correlation id e command sem reply	8	**8 / 8**
Command sem reply que falha → `prompt.errors`	3	**3 / 3**
Query sem reply (o app segue saudável)	1	**1 / 1**
Total	**43**	**43 / 43 · 0 falhas**

8.3 Outras verificações

- **Concorrência:** 40 comandos simultâneos (de dois subjects diferentes) sobre o mesmo prompt geraram as versões 2 a 41, sem perda nem duplicata.
- **Persistência e ciclo de vida:** os dados sobreviveram a um reinício (arquivo em modo WAL) e o SIGTERM encerrou o app de forma limpa.
- **Timeout:** com o limite forçado a 0,0001 s, o cliente recebeu `timeout` e o log mostrou o traceback formatado.
- **Isolamento:** nenhuma menção a infraestrutura, adapters ou bibliotecas em `domain/`, `app/` e `port/`; pyright com 0 erros.

8.4 Lições do caminho

O que deu errado	O que aprendemos
O SQLAlchemy 2.1 pediu o *greenlet* ao ser usado de forma assíncrona.	Ler a mensagem de erro até o fim: ela dizia o extra a instalar (`sqlalchemy[asyncio]`).
O traceback saiu como um dicionário bruto no log.	Duas bibliotecas boas podem conflitar na junção; testar o caminho de erro, não só o feliz.
O teste de fumaça falhava de modo intermitente com `not_found`.	Havia **duas instâncias do app** no mesmo NATS, e as duas respondiam (cada uma com seu banco). Sem um *queue group*, todos os assinantes recebem tudo.
restore funcionava em prompt não apagado.	O teste de ponta a ponta achou um buraco de regra de negócio que a leitura do código não mostrou.

ATENÇÃO · Escalando para várias instâncias

Hoje, duas instâncias do app ligadas ao mesmo NATS **executam cada comando duas vezes**. Para escalar horizontalmente, as assinaturas precisam de um *queue group* (o NATS entrega cada mensagem a apenas um membro do grupo), e o banco compartilhado precisaria de controle de concorrência além do lock local. É um próximo passo natural, ainda não feito.

RESUMO DO CAPÍTULO

- Cada camada foi testada no nível adequado; o **smoke test** valida o sistema pela porta da frente: 43 de 43.
- Os melhores aprendizados vieram de testes que **falharam** — vale agradecê-los.

9 Mão na massa

Rodar, explorar e estender o projeto por conta própria

Pergunta-guia: *como eu mesmo uso o sistema, e como o estendo sem quebrar a arquitetura?*

9.1 Subindo o ambiente

Terminal

```
# 1) o servidor NATS (em um terminal; precisa do Docker)
docker compose up -d

# 2) o Prompt Vault (em outro terminal, dentro da pasta do projeto)
uv sync
uv run src/main.py

# 3) a verificação completa
./smoke_test.sh
```

9.2 Receitas com o NATS CLI

Substitua `<ID>` pelo `prompt_id` devolvido ao criar o prompt. Abra um segundo terminal rodando `nats sub 'prompt.events.>'` para ver os eventos chegando.

Receitas

```
# criar um prompt
nats request prompt.create '{"name":"saudacao","content":"Ola\nmundo"}' --raw

# nova versão (não ativa) e alteração de conteúdo (cria e ativa)
nats request prompt.version.create '{"prompt_id":"<ID>","content":"Oi\nmundo"}' --raw
nats request prompt.content.update '{"prompt_id":"<ID>","content":"E ai\nmundo\nfim"}' --raw

# consultar, comparar e voltar atrás
nats request prompt.get '{"prompt_id":"<ID>"}' --raw
nats request prompt.versions.compare '{"prompt_id":"<ID>","from_version":1,"to_version":3}' --raw
nats request prompt.version.activate '{"prompt_id":"<ID>","version_number":1}' --raw

# apagar (soft) e recuperar; listar
nats request prompt.delete '{"prompt_id":"<ID>"}' --raw
nats request prompt.list.deleted '{}' --raw
nats request prompt.restore '{"prompt_id":"<ID>"}' --raw

# comando sem resposta, com seu próprio código de rastreio
nats pub prompt.delete '{"prompt_id":"<ID>"}' -H "X-Correlation-Id:meu-rastreio"
```

9.3 Exercícios

#	Exercício	Dica
1	**Observe os eventos.** Com `nats sub 'prompt.events.>'` aberto, crie um prompt e altere seu conteúdo. Quais eventos aparecem, em que ordem?	Procure o cabeçalho `X-Correlation-Id` na saída.

#	Exercício	Dica
2	**Leia um diff.** Crie uma versão 2 com uma linha alterada, uma removida e uma acrescentada e compare as versões. Classifique cada entrada.	Os tipos são `added`, `removed`, `modified`, `unchanged`.
3	**Provoque os três tipos de erro:** JSON inválido, UUID inexistente e restaurar um prompt ativo. Que `code` vem em cada resposta?	Respostas: `invalid_payload`, `not_found`, `domain_error`.
4	**Siga um correlation id.** Envie um pedido com `-H "X-Correlation-Id:meu-rastreio"` e procure esse código na saída do app.	`grep` ou o recurso de busca do terminal.
5	**Desafio (domain):** escreva uma `DiffStrategy` que compare *palavra por palavra*.	Crie uma classe em `src/domain/diff/` com `name` e `diff`; troque a estratégia no `main.py`. Que arquivos **não** precisaram mudar?
6	**Desafio (nova função):** "renomear prompt". Planeje, camada por camada, o que mudaria.	Veja o plano abaixo — e conte quantas camadas são tocadas.

NO PROJETO · Exercício 6: o mapa da mudança

1. **domain:** método `Prompt.rename(name)` (valida nome não vazio, atualiza a data) e um evento `PromptRenamed`.
2. **port:** *nada muda* — salvar um prompt alterado já é o `save` de sempre.
3. **app:** `RenamePromptUseCase` e `RenamePromptCommand`, no mesmo molde de `DeletePromptUseCase`.
4. **adapter/messaging:** o subject `prompt.rename`, um handler e uma `Route` em `build_routes`; o evento ganha seu nome em `_EVENT_NAMES`.
5. **adapter/persistence:** *nada muda* — o mapper já copia o campo `name` (`apply_state`).
6. **smoke_test.sh:** novas verificações para o subject e o evento.

Repare: o banco e as portas ficam intactos. É a arquitetura mostrando o seu valor — a mudança toca só onde ela faz sentido.

9.4 Próximos passos naturais

Melhoria	Por quê
Queue group nas assinaturas	Permitir várias instâncias do app sem executar cada comando em duplicidade.
Padrão outbox	Gravar o evento na mesma transação do dado, para nunca perder um evento publicado depois do commit.
Porta de leitura (`PromptReader`)	Separar leitura de escrita nas portas e eliminar a concessão do repositório de leitura.
Testes permanentes (pytest)	Transformar os scripts de verificação em uma suíte que roda a cada mudança.
Paginação nas listagens	Não devolver todos os prompts de uma vez quando houver muitos.
Documentação AsyncAPI	Descrever formalmente subjects e payloads, para outros times integrarem.

A

Glossário

Os termos do guia em linguagem simples

Termo	Significado	Termo	Significado
Adapter	Peça que traduz entre o núcleo e uma tecnologia concreta (NATS, SQLite).	**NATS**	Sistema de mensagens baseado em subjects, com publicar/assinar e pedido/resposta.
Agregado	Grupo de objetos tratado como unidade, com uma raiz que protege as regras (Prompt + versões).	**ORM**	Mapeia tabelas de banco para classes da linguagem (SQLAlchemy).
Assíncrono	Que não fica parado esperando: enquanto uma tarefa espera, outra avança (async/await).	**Outbox**	Padrão que grava eventos junto com os dados para nunca perdê-los.
Broker	O servidor central que recebe e distribui mensagens (o servidor NATS).	**Port**	Contrato (Protocol) que o núcleo exige do mundo externo.
Command	Caso de uso que altera o estado do sistema.	**Prompt**	Texto de instrução enviado a um modelo de linguagem.
Composition root	O único lugar que cria e liga todas as peças (main.py).	**Protocol**	Descrição de métodos que uma classe precisa ter, sem herança.
Context manager	Estrutura `with` que garante abrir e fechar algo corretamente.	**Query**	Caso de uso que apenas lê.
Correlation id	Etiqueta de rastreio que acompanha uma mensagem em respostas, eventos e logs.	**Request / reply**	Conversa em que quem pergunta espera a resposta.
Dataclass	Classe cujo construtor e comparações são gerados automaticamente.	**Rollback**	Desfazer tudo que uma transação fez.
Dependency injection	Entregar a um objeto as peças de que ele precisa, em vez de ele criá-las.	**Soft delete**	Apagar “escondendo”: o dado continua guardado e é recuperável.
Diff	Comparação que mostra o que mudou entre dois textos.	**Strategy**	Padrão para trocar um algoritmo sem mexer em quem o usa.
Domain event	Fato ocorrido no domínio, imutável e no passado (“versão ativada”).	**Subject**	O “assunto” (canal) onde as mensagens do NATS são publicadas.
Duck typing	“Se anda e grasna como um pato…”: vale a forma do objeto, não a herança.	**Timeout**	Tempo máximo de espera antes de desistir.
Envelope	O formato padrão de resposta: ok, correlation_id e data ou error.	**Transação**	Conjunto de operações do tipo “tudo ou nada”.
Hexagonal	Arquitetura de “portas e adaptadores” que isola o núcleo da tecnologia.	**Type hint**	Anotação do tipo esperado de um valor (`str | None`).
Invariante	Afirmação que nunca pode ser falsa (“versões numeradas sem buracos”).	**Unit of Work**	Objeto que gerencia uma transação e confirma ou desfaz tudo.
LLM	Modelo de linguagem de grande porte; recebe prompts e gera texto.	**uv**	Ferramenta que cria o ambiente Python e instala dependências.
Mapper	Código que traduz entre o modelo do domínio e o modelo das tabelas.	**WAL**	Modo do SQLite em que leituras e escrita convivem, via um “rascunho” de gravações.

B Mapa de arquivos

Onde está cada coisa, e a que camada pertence

Arquivo	Camada	Responsabilidade
`domain/entities/prompt.py`	**domain**	Prompt: o agregado raiz, com todas as regras e eventos.
`domain/entities/prompt_version.py`	**domain**	PromptVersion: snapshot imutável do conteúdo.
`domain/status.py`	**domain**	PromptStatus e a tabela de transições permitidas.
`domain/events.py`	**domain**	Os sete eventos de domínio.
`domain/exceptions.py`	**domain**	Erros de negócio (DomainError e filhos).
`domain/diff/models.py`	**domain**	ChangeType, DiffEntry e VersionDiff.
`domain/diff/strategy.py · line_diff.py`	**domain**	DiffStrategy (contrato) e LineDiffStrategy (linha a linha, com difflib).
`port/*.py`	**port**	Os três contratos: PromptRepository, EventPublisher e UnitOfWork.
`app/commands/*.py`	**app**	Seis casos de uso de escrita.
`app/queries/*.py`	**app**	Cinco casos de uso de leitura (incluindo CompareVersions).
`app/exceptions.py · app/_helpers.py`	**app**	PromptNotFoundError e require_prompt.
`adapter/persistence/models.py`	**adapter**	Tabelas ORM e o tipo de data UTC.
`adapter/persistence/mapper.py`	**adapter**	Tradução domínio ⇄ tabelas.
`adapter/persistence/prompt_repository.py`	**adapter**	Repositório com SQLAlchemy (escrita e leitura).
`adapter/persistence/unit_of_work.py`	**adapter**	Transação concreta, com a chave de escrita.
`adapter/messaging/subjects.py`	**adapter**	Nomes dos subjects.
`adapter/messaging/codec.py`	**adapter**	JSON ⇄ objetos, validação de campos e envelopes.
`adapter/messaging/handlers.py`	**adapter**	Tradução subject → caso de uso.
`adapter/messaging/server.py`	**adapter**	Assinaturas, timeout, respostas e logs.
`adapter/messaging/event_publisher.py`	**adapter**	Publica os eventos de domínio no NATS.
`adapter/messaging/correlation.py`	**adapter**	Correlation id por tarefa assíncrona.
`adapter/messaging/errors.py · serializers.py`	**adapter**	Exceção → código; objetos → dicionários.
`infra/settings.py`	**infra**	Configuração por variáveis de ambiente.
`infra/database.py`	**infra**	Motor SQLite, ajustes de concorrência, criação das tabelas.
`infra/nats_client.py`	**infra**	Conexão com o NATS e seus eventos.
`infra/logging.py`	**infra**	structlog + rich.
`main.py · docker-compose.yml · smoke_test.sh · pyproject.toml · uv.lock`	—	Raiz de composição · NATS local · teste de fumaça · dependências.

C Para ir além

Leituras e documentações para continuar de onde este guia parou

Livros e artigos

Obra	Por que ler
Hexagonal Architecture — Alistair Cockburn (artigo, 2005)	A origem da ideia de portas e adaptadores, pelo próprio autor.
Domain-Driven Design — Eric Evans (2003)	A fonte de agregados, entidades, eventos e "modelo rico".
Architecture Patterns with Python — Harry Percival e Bob Gregory (2020)	Repository, Unit of Work, eventos e mensageria, tudo em Python. Muito próximo deste projeto.
Patterns of Enterprise Application Architecture — Martin Fowler (2002)	Descrição clássica de Repository e Unit of Work.

Documentação oficial

Tecnologia	Onde procurar
Python (asyncio, dataclasses, typing)	docs.python.org
uv	docs.astral.sh/uv
NATS	docs.nats.io
SQLAlchemy 2.x (ORM e asyncio)	docs.sqlalchemy.org
structlog	structlog.org
rich	rich.readthedocs.io

CONCEITO · Como estudar a partir daqui

Escolha uma pasta (comece pelo `domain/`), leia os arquivos de cima para baixo e, a cada função, pergunte: *"quem a chama? o que ela protege?"*. Depois mude algo pequeno, rode o `smoke_test.sh` e veja o que acontece. Código se aprende lendo, rodando e quebrando de propósito.

D Respostas das pausas

As perguntas de "Teste o que aprendeu", respondidas

Cap.	Pergunta	Resposta
2	Para que serve o `uv.lock`?	Registra as versões exatas de tudo que foi instalado, para reproduzir o mesmo ambiente em qualquer máquina. É gerado pelo uv; editar à mão arrisca desfazer essa garantia.
2	O que é `int | None`?	Um valor que pode ser um número inteiro **ou** nada (None). Em `active_version_number`, None significa "nenhuma versão ativa".
2	E se houver erro antes do `commit()` num `async with uow`?	Ao sair do bloco, o `__aexit__` faz *rollback*: nada que foi feito na transação fica gravado.
3	Por que o domain não importa `sqlalchemy`?	Para ficar independente de tecnologia: assim é testável sem banco e as regras sobrevivem à troca de tecnologia.
3	O que alterar para trocar SQLite por PostgreSQL?	`adapter/persistence` e a configuração em `infra/database.py` (URL e driver). Domain, app e port permanecem intactos.
3	Porta × adapter?	Porta é o **contrato** (um Protocol); adapter é a **implementação concreta** desse contrato, ligada a uma tecnologia.
4	Por que `PromptVersion` é `frozen`?	Porque o histórico é imutável: uma versão, uma vez criada, nunca muda. Mudar de versão ativa só move o ponteiro.
4	O que faz `restore()` num prompt ativo?	Lança `PromptNotDeletedError`: só se restaura o que está apagado.
4	Diff palavra por palavra?	Uma nova classe com `name` e `diff(old, new)` (uma `DiffStrategy`), entregue ao caso de uso no `main.py`.
5	Command × query?	Command altera o estado (usa transação e publica eventos); query só lê (não abre transação nem publica nada).
5	Por que publicar depois do commit?	Para não anunciar ao mundo algo que, por um erro na gravação, acabou não acontecendo.
5	Por que receber o `uow` no construtor?	Injeção de dependência: o caso de uso não sabe como a transação é feita, o que permite trocá-la (inclusive por um falso, em testes).
6	Para que serve o correlation id?	Para rastrear uma mensagem do início ao fim: ele aparece na resposta, nos eventos e em cada linha de log.
6	Por que uma instância nova de cada caso de uso por mensagem?	Cada um carrega seu Unit of Work e sua sessão de banco, que não podem ser compartilhados por mensagens simultâneas.
6	O que o WAL melhora?	Leituras e escrita convivem: leitores não ficam bloqueados enquanto alguém grava.